

Session 22:

GUI (Graphical User Interface)

Dr. Naveen Kumar
Associate Professor
Amity Institute of Information Technology
Amity University Patna

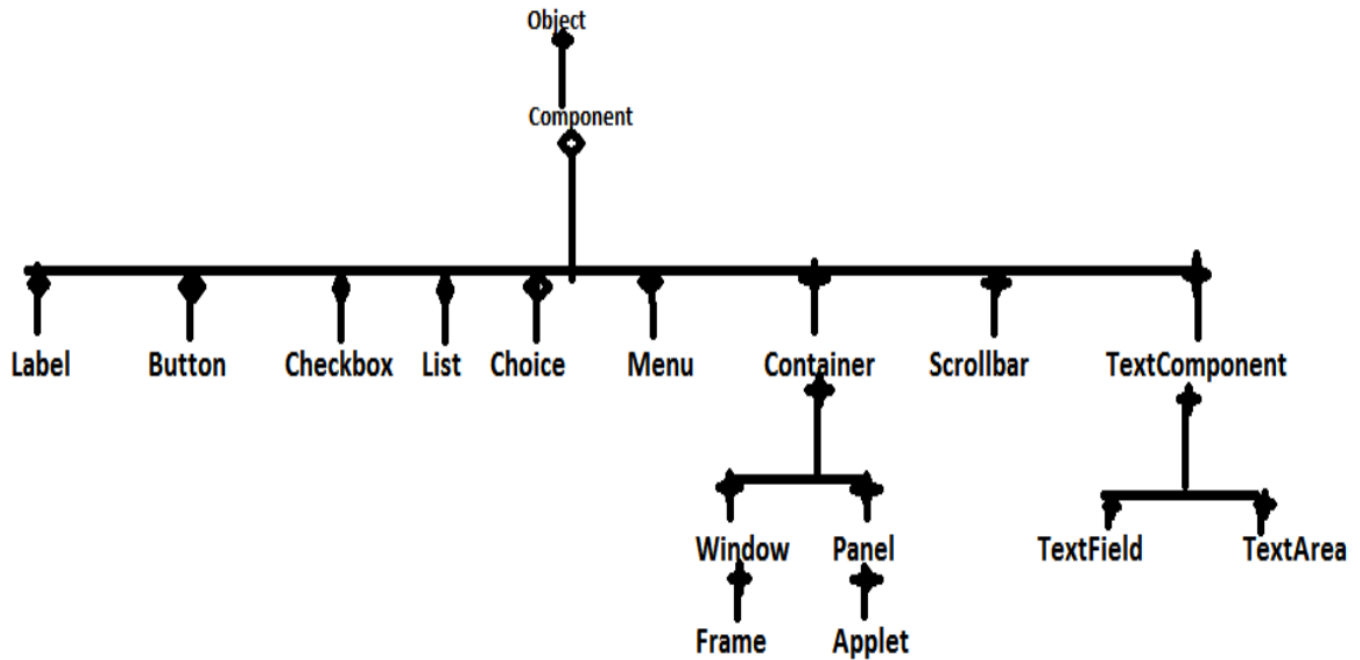
Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

- Java.awt is a package provides environment to develop graphical user interface applications.
- AWT means (**Abstract Window Toolkit**). AWT is used to prepare the components but it is not providing any life to that components means by using AWT it is possible to create a static components.
- To provide the life to the static components we need some other package is called **java.awt.event** package.
- By using AWT we are preparing application these applications are called console based or CUI application.
- **Java.awt package** is used to prepare static components.
- **Java.awt.event package** is used to provide the life to the static components.

- GUI(graphical user interface):-
- 1. It is a mediator between end user and the program.
- 2. AWT is a package it will provide predefined support to design GUI applications.
- **Component** : - Component is an object which is displayed pictorially on the screen.
- Ex:- Button, Label, TextField.....etc
- **Container**:- Container is a GUI component, it is able to accommodate all other GUI components.
- Ex:- Frame, Applet.
- **Event**:- The event is action generated on the component or the change is made on the state of the object.
- Ex:- Button clicked, Checkboxchecked, Itemselected in the list, Scrollbar scrolled horizontal/vertically.

Classes of AWT

- Classes of AWT:- The classes present in the AWT package.



Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

Frame Class

- Frame is a class which is present in java.awt package.
- Frame is a Basic component in AWT, because all the components displayed in a Frame.
- We are displaying pictures on the Frame.
- It is possible to display some text on the Frame. Based on the above reasons the frame will become basic component in AWT.

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

Constructors

- create a Frame class object.
- `Frame f=new Frame();`
- create a Frame class object and pass frame title
- `Frame f=new Frame("MyFrame");`
- Take a subclass to the Frame and create object to the subclass.
- `class MyFrame extends Frame`
- `MyFrame f=new MyFrame();`

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

Characteristics of the Frame

- When we create a Frame class object the Frame will be created automatically with the invisible mode.
- To provide visible mode to following method.
- **public void setVisible(boolean b)**
- where b==true means visible mode.
- where b==false means invisible mode.
- Ex: **f.setVisible(true);**
- When we created a Frame, the initial size of the Frame is : 0 pixel height 0 pixel width So it is not visible to use.
- To provide particular size to the Frame we have to use following method.
- **public void setSize(int width,int height)**
- Ex: **f.setSize(400,500);**

- To provide title to the Frame explicitly we have to use the following method
- `public void setTitle(String Title)`
- Ex: `f.setTitle("MyFrame");`
- When we create a Frame, the default background color of the Frame is white.
- If you want to provide particular color to the Frame we have to use the following method.
- `public void setBackground(color c)`
- Ex: `f.setBackground(Color.red);`

CREATION OF FRMAE

```
import java.awt.*;
class Demo
{
    public static void main(String[] args)
    {
        //frame creation
        Frame f=new Frame();
        //set visibility
        f.setVisible(true);
        //set the size of the frame
        f.setSize(400,400);
        //set the background
        f.setBackground(Color.red);
        //set the title of the frame
        f.setTitle("myframe");
    }
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

CRATION OF FRAME BY TAKING USER DEFINED CLASS

```
import java.awt.*;
class MyFrame extends Frame
{
    MyFrame()
    {
        setVisible(true);
        setSize(500,500);
        setTitle("myframe");
        setBackground(Color.red);
    }
}

class Demo
{
    public static void main(String[] args)
    {
        MyFrame f=new MyFrame();
    }
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

display text on the screen

- 1. If you want to display some textual message or some graphical shapes on the Frame then we have to override `paint()`, which is present in the `Frame` class.
- `public void paint(Graphics g)`
- 2. To set a particular font to the text, we have to use **Font** class present in `java.awt` package
- `Font f=new Font(String type,int style,int size);`
- Ex: `Font f= new Font("arial",Font.Bold,30);`

```
import java.awt.*;
class Test extends Frame
{
    public static void main(String[] args)
    {
        Test t=new Test();
        t.setVisible(true);
        t.setSize(500,500);
        t.setTitle("Strudent Registration");
        t.setBackground(Color.red);
    }
    public void paint(Graphics g)
    {
        Font f=new Font("arial",Font.ITALIC,25);
        g.setFont(f);
        g.drawString("hi Welcome to Amity University, how r u",100,100);
    }
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

- Note:- 1. When we create a MyFrame class constructor, jvm executes MyFrame class constructor just before this JVM has to execute Frame class zero argument constructor.
- 2. In Frame class zero argument constructor repaint() method will be executed, it will access predefined Frame class paint() method.
- But as per the requirement overriding paint() method will be executed.
- 3. Therefore the paint() will be executed automatically at the time of Frame creation.

Preparation of the components

- **Label:** - 1) Label is a constant text which is displayed along with a TextField or TextArea.
- 2) Label is a class which is present in java.awt package.
- 3) To display the label we have to add that label into the frame for that purpose we have to use add() method present in the Frame class.
- Constructor:-
- **Label l=new Label();**
- **Label l=new Label("user name");**

```
import java.awt.*;
class Test
{
    public static void main(String[] args)
    {
        Frame f=new Frame();
        f.setVisible(true);
        f.setTitle("AIIT – Amity University");
        f.setBackground(Color.red);
        f.setSize(400,500);
        Label l=new Label("user name:");
        f.add(l);
    }
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

TextField

- 1) TextField is an editable area.
- 2) In TextField we are able to provide single line of text.
- 3) Enter Button doesn't work on TextField.
- To add TextField into the Frame we have to use **add() method**.
- 1. To set Text to the text area we have to use the following method.
- **t.setText("AIIT (Amity Institute of Information Technology)");**
- 2. To get the text from the TextArea we have to use following method.
- **String s=t.getText();**
- 3. To append the text into the TextArea.
- **t.appendText("ratan");**
- Constructor:- **TextFiled tx=new TextFiled();**
- **TextField tx=new TextField("ratan");**


```
import java.awt.*;
class Test
{
    public static void main(String[] args)
    {
        Frame f=new Frame();
        f.setVisible(true);
        f.setTitle("Name: ");
        f.setBackground(Color.red);
        f.setSize(400,500);
        //TextField tx=new TextField(); empty TextField
        TextField tx=new TextField("Dr. Naveen Kumar");
        //TextField with data
        f.add(tx);
    }
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

TextArea

- 1) TextArea is a class present in java.awt.package.
- 2) TextArea is a Editable Area. Enter button will work on TextArea.
- 3) To add the TextArea into the frame we have to use the **add()**
- Construction:- **TextArea t=new TextArea();**
- **TextArea t=new TextArea(int rows, int columns);**
- 4. To set Text to the textarea we have to use the following method.
- **t.setText("Amity University");**
- 5. To get the text form the TextArea we have to use fallowing method. **String s=t.getText();**
- 6. To append the text into the t.appendText("AIIT");

```
import java.awt.*;
class Test
{
    public static void main(String[] args)
    {
        Frame f=new Frame();
        f.setVisible(true);
        f.setTitle("AIIT");
        f.setBackground(Color.red);
        f.setSize(400,500);
        f.setLayout(new FlowLayout());
        Label l=new Label("user name:");
        TextArea tx=new TextArea(4,10);//4 character height 10 character width
        tx.appendText("AIIT");
        tx.setText("BCA/MCA");
        System.out.println(tx.getText());
        f.add(l);
        f.add(tx);
    }
}
```

Choice

- 1) Choice is a class present in java.awt package.
- 2) List is allows to select multiple items but choice is allow to select single Item.
- Constructor:- **Choice ch=new Choice();** Methods :-
- 1. To add items to the choice we have to use following method.
- ch.add("HYD");
- ch.add("Chennai");
- ch.add("BANGALORE");
- 2. To remove item from the choice based on the string.
- ch.remove("HYD");
- ch.remove("BANGALORE");

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

- 3. To remove the item based on the index position **ch.remove(2);**
- 4. To remove the all elements **ch.removeAll();**
- 5. To inset the data into the choice based on the particular position. **ch.insert(2,"ratan");**
- 6. To get selected item from the choice we have to use following method.
- **String s=ch.getSelectedItem();**
- 7. To get the selected item index number we have to use fallowing method
- **int a=ch.getSelectedIndex();**

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

```
import java.awt.*;
class Test
{
    public static void main(String[] args)
    {
        Frame f=new Frame();
        f.setVisible(true);
        f.setTitle("Amity University");
        f.setBackground(Color.red);
        f.setSize(400,500);
        Choice ch=new Choice();
        ch.add("c");
        ch.add("cpp");
        ch.add("java");
        ch.add(".net");
        ch.remove(".net");
        ch.remove(0);
        ch.insert("Adv. Java",0);
        f.add(ch);
        System.out.println(ch.getItem(0));
        System.out.println(ch.getSelectedItem());
        System.out.println(ch.getSelectedIndex());
        //ch.removeAll();
    }
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

List

- 1) List is a class it is present in java.awt.package
- 2) List is providing list of options to select. Based on your requirement we can select any number of elements. To add the List to the frame we have to use add() method.
- **CONSTRUCTOR:-**
- 1) List l=new List();
- It will creates the list by default size is four elements. And it is allow selecting the only one item at a time.
- 2) List l=new List(3); It will display the three items size and it is allow selecting the only single item.
- 3) List l=new List(5,true); It will display the five items and it is allow selecting the multiple items.

- Methods:- 1. To add the elements to the List we have to use following method.
 - **l.add("c");**
 - l.add("cpp");
 - l.add("java");
 - l.add("ratan",0);
- 2. To remove element from the List we have to use following method.
 - l.remove("c");
 - l.remove(2);
- 3. To get selected item from the List we have to use following method. **String x=l.get SelectedItem();**
- 4. To get selected items from the List we have to use following method. **String[] x=s.get SelectedItems()**


```
import java.awt.*;
class Test
{
    public static void main(String[] args)
    {
        Frame f=new Frame();
        f.setVisible(true);
        f.setTitle("Amity University");
        f.setBackground(Color.red);
        f.setSize(400,500);
        f.setLayout(new FlowLayout());
        List l=new List(4,true);
        l.add("c");
        l.add("cpp");
        l.add("java");
        l.add(".net");
        l.add("ratan");
        l.add("arun",0);
        l.remove(0);
        f.add(l);
        System.out.println(l.getSelectedItem());
    }
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

Checkbox

- 1) Checkbox is a class present in java.awt package.
- 2) The user can select more than one checkbox at a time. To add the checkbox to the frame we have to use add() method.

Constructor:-

- 1) `Checkbox cb1=new CheckBox(); cb1.setLabel("BTECH");`
- 2) `Checkbox cb1=new CheckBox("MCA");`
- 3) `Checkbox cb3=new CheckBox("BSC",true);`

CheckBox Methods

1. To set a label to the CheckBox explicitly and to get label from the CheckBox we have to use the
2. following method. `cb.setLabel("BSC");`
3. To get the label of the checkbox we have to use following method. `String str=cb.getLabel();`
4. To get state of the CheckBox and to set state to the CheckBox we have to use following method. `Boolean b=ch.getState();`

```
import java.awt.*;
class Test {
public static void main(String[] args) {
Frame f=new Frame();
f.setVisible(true);
f.setTitle("ratan"); f.setBackground(Color.red); f.setSize(400,500);
Checkbox cb1=new Checkbox("BTECH",true); f.add(cb1);
System.out.println(cb1.getLabel());
System.out.println(cb1.getState());
}
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

RADIO BUTTON

- 1) AWT does not provide any predefined support to create RadioButtons.
- 2) It is possible to select Only item is selected from group of item. To add the RadioButton to the frame we have to use add() method.

By using two classes we create Radio Button those are

a)CheckBoxgroup b)CheckBox

step 1:- Create CheckBox group object. `CheckBoxGroup cg=new CheckBoxGroup();`

step 2:- pass Checkbox object to the CheckboxGroup class then the radio buttons are created. `CheckBox cb1=new CheckBox("male",cg,false);`
`CheckBox cb2=new CheckBox("female",cg,false);`

Methods:-

1) To set status and to get status we have to use setState() and getState() methods.

```
String str=Cb.getState();
```

```
Cb.setState();
```

2) To get Label and to set Label we have to use following methods.

```
String str=getLabel() setLabel("female").
```

```
import java.awt.*;
class Test
{
    public static void main(String[] args) {
        Frame f=new Frame();
        f.setVisible(true);
        f.setTitle("ratan");
        f.setBackground(Color.red);
        f.setSize(400,500);
        CheckboxGroup cg=new CheckboxGroup();
        Checkbox cb1=new Checkbox("male",cg,true);
        f.add(cb1);
        System.out.println(cb1.getLabel());
        System.out.println(cb1.getState());
    }
}
```

Layout Managers

```
import java.awt.*;
class Test {
public static void main(String[] args) {
Frame f=new Frame();
f.setVisible(true);
f.setTitle("ratan");
f.setBackground(Color.red);
f.setSize(400,500);
Label l1=new Label("user name:");
TextField tx1=new TextField();
Label l2=new Label("password:");
TextField tx2=new TextField();
Button b=new Button("login");
f.add(l1);
f.add(tx1);
f.add(l2);
f.add(tx2);
f.add(b);
}
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

Event delegation model

1. When we create a component the components visible on the screen but it is not possible to perform any action for example button.
 2. Whenever we create a Frame it can be minimized and maximized and resized but it is not possible to close the Frame even if we click on Frame close Button.
 3. The Frame is a static component so it is not possible to perform actions on the Frame.
 4. To make static component into dynamic component we have to add some actions to the Frame.
 5. To attach actions to the Frame component we need event delegation model.
- **Note: Whenever we click on button no action will be performed clicking like this is called event.**

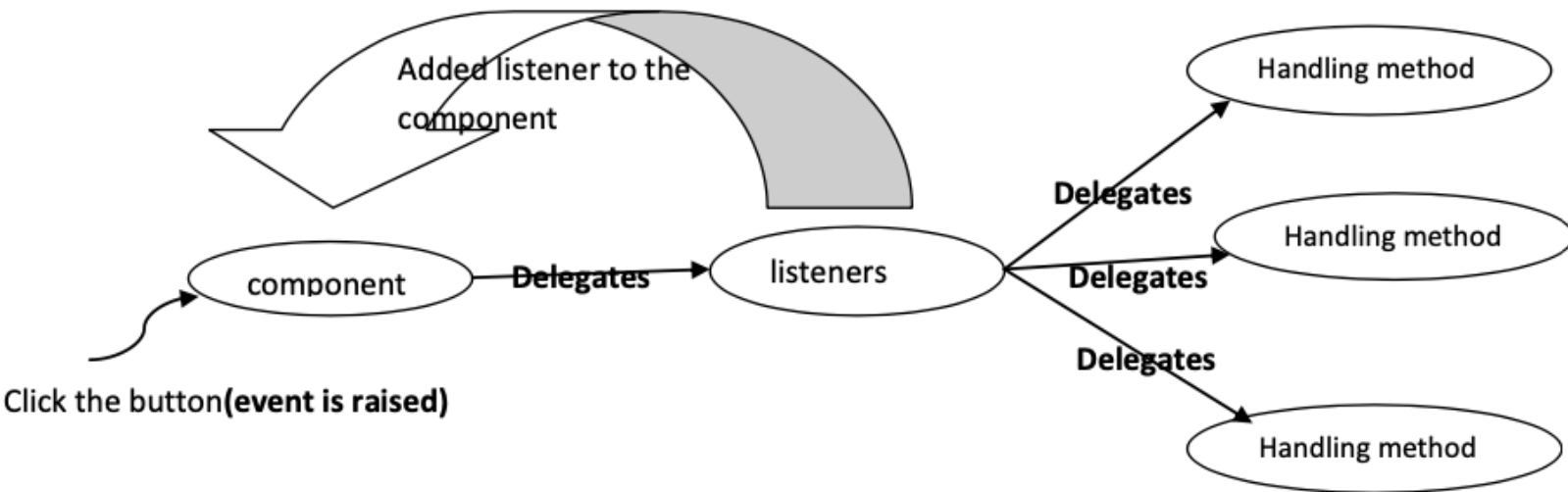
Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

Event

Event is nothing but a particular action generated on the particular component.

1. When an event generates on the component the component is unable to respond because component can't listen the event.
2. To make the component listen the event we have to add listeners to the component.
3. Wherever we are adding listeners to the component the component is able to respond based on the generated event.
4. A listener is an interface which contain abstract methods and it is present in java.awt.event package
5. The listeners are different from component to component.

A component delegate event to the listener and listener is designates the event to appropriate method by executing that method only the event is handled. This is called **Event Delegation Model**.



Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

- Note: -
To attach a particular listener to the Frame we have to use following method
- `Public void AddxxxListener(xxxListener e)`
Where xxx may be ActionListener, windowListener
The Appropriate Listener for the Frame is “windowListener”

class MyFrame **extends** Frame **implements** ActionListener {

TextField tx1,tx2,tx3;

Label l1,l2,l3;

Button b1,b2;

int result;

MyFrame() {

this.setSize(250,400);

this.setVisible(**true**);

this.setLayout(**new** FlowLayout());

l1=**new** Label("first value");

l2=**new** Label("second value");

l3=**new** Label("result");

tx1=**new** TextField(25);

tx2=**new** TextField(25);

tx3=**new** TextField(25);

b1=**new** Button("add");

b2=**new** Button("mul");

b1.addActionListener(**this**);

b2.addActionListener(**this**);

this.add(l1);

this.add(tx1);

this.add(l2);

this.add(tx2);

this.add(l3);

this.add(tx3);

this.add(b1);

this.add(b2);

}

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

```
public void actionPerformed(ActionEvent e) {
    try{
        int fval=Integer.parseInt(tx1.getText());
        int sval=Integer.parseInt(tx2.getText());
        String label=e.getActionCommand();
        if (label.equals("add")) {
            result=fval+sval;
        }
        if (label.equals("mul")) {
            result=fval*sval;
        } tx3.setText(" "+result); }
        catch(Exception ee) {
            ee.printStackTrace();
        }
    }
}

class Demo {
    public static void main(String[] args) {
        MyFrame f=new MyFrame();
    }
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

ScrollBar

- 1. ScrollBar is a class present in the java.qwt.package
- 2. By using ScrollBar we can move the Frame up and down.
- ScrollBar s=new ScrollBar(int type)
- Type of scrollbar
 1. VERTICAL ScrollBar
 2. HORIZONTAL ScrollBar
- To create a HORIZONTAL ScrollBar:-
ScrollBar sb=new ScrollBar(ScrollBar.HORIZONTAL);
To get the current position of the scrollbar we have to use the following method. public int getValue()
- To create a VERTICAL ScrollBar:-
ScrollBar sb=new ScrollBar(ScrollBar.VERTICAL);

Appropriate Listeners for Components

Component	Event Name	Listener Name	Listener Methods
Frame	Window Event	Window Listener	p v WindowOpened(WE) p v WindowActivated(WE) p v WindowDeactivated(WindowEvent e) p v WindowClosing(WindowEvent e) p v WindowClosed(WindowEvent e) p v WindowIconified(WindowEvent e) p v WindowDeiconified(WindowEvent e)
TextField	ActionEvent	ActionListener	P v ActionPerformed(AE)
TextArea	ActionEvent	ActionListener	P v ActionPerformed(AE)
Menu	ActionEvent	ActionListener	P v ActionPerformed(AE)
Button	ActionEvent	ActionListener	P v ActionPerformed(AE)
Checkbox	ItemEvent	ItemListener	P v ItemStateChanged(ItemEvent)
Radio	ItemEvent	ItemListener	P v ItemStateChanged(ItemEvent)
List	ItemEvent	ItemListener	P v ItemStateChanged(ItemEvent)

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

Appropriate Listeners for Components

Component	Event Name	Listener Name	Listener Methods
Choice	ItemEvent	ItemListener	P v ItemStateChanged(ItemEvent)
Scrollbar	AdjustmentEvent	AdjustmentListener	Pv AdjustmentValueChanged(AdjustmentEvent)
Mouse	MouseEvent	MouseListener	pv MouseEntered(MouseEvent e) pv MouseExited(MouseEvent e) pv MousePressed(MouseEvent e) pv MouseReleased(MouseEvent e) pv MouseClicked(MouseEvent e)
Keyboard	KeyEvent	KeyListener	pv KeyTyped(KeyEvent e) pv KeyPressed(KeyEvent e) pv KeyReleased(KeyEvent e)
		Dr. Naveen Kumar, Associate Professor,	
		AIIT, AUP	

PROVIDING CLOSING OPTION TO THE FRAME

```
import java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame {
    MyFrame() {
        this.setSize(400,500);
        this.setVisible(true);
        this.setTitle("myframe");
        this.addWindowListener(new MyClassImpl());
    }
}
```

PROVIDING CLOSING OPTION TO THE FRAME

```
class MyClassImpl implements WindowListener{
    public void windowActivated(WindowEvent e) {
        System.out.println("window activated");
    }
    public void windowDeactivated(WindowEvent e) {
        System.out.println("window deactivated");
    }
    public void windowIconified(WindowEvent e) {
        System.out.println("window iconified");
    }
    public void windowDeiconified(WindowEvent e) {
        System.out.println("window deiconified");
    }
    public void windowClosed(WindowEvent e) {
        System.out.println("window closed");
    }
    public void windowClosing(WindowEvent e) {
        System.exit(0);
    }
    public void windowOpened(WindowEvent e) {
        System.out.println("window Opened");
    }
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

PROVIDING CLOSING OPTION TO THE FRAME

```
class Demo{  
    public static void main(String[] args) {  
        MyFrame f=new MyFrame();  
    }  
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

PROVIDING CLOSEING OPTION THE FRAME

```
import java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame {
    MyFrame() {
        this.setVisible(true);
        this.setSize(500,500);
        this.setBackground(Color.red);
        this.setTitle("rattaiah");
        this.addWindowListener(new ListenerImpl());
    }
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

```
class ListenerImpl extends WindowAdapter {  
public void windowClosing(WindowEvent we) {  
    System.exit(0);  
}  
}
```

```
class Demo {  
public static void main(String[] args) {  
    MyFrame f=new MyFrame();  
}  
}
```

Note :- by using WindowAdaptor class we can close the frame. Internally WindowAdaptor class implements WindowListener interface. Hence WindowAdaptor class contains empty implementation of abstract methods.

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

PROVIDING CLOSEING OPTION THE FRAME

```
import java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame {
    MyFrame() {
        this.setVisible(true);
        this.setSize(500,500);
        this.setBackground(Color.red);
        this.setTitle("rattaiah");
        this.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent we) {
                System.exit(0);
            }
        });
    }
}
```

- **class** FrameEx {
- **public static void** main(String[] args) {
- MyFrame f=new MyFrame();
- }
- }

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

WRITE SOME TEXT INTO THE FRAME

```
import java.awt.*;
class MyFrame extends Frame
{
    MyFrame() {
        this.setVisible(true);
        this.setSize(500,500);
        this.setBackground(Color.red);
        this.setTitle("rattaiah");
    }
    public void paint(Graphics g) {
        Font f=new Font("arial",Font.BOLD,20);
        g.setFont(f);
        this.setForeground(Color.green);
        g.drawString("HI BTECH ",100,100);
        g.drawString("good boys &",200,200);
        g.drawString("good girls",300,300); }
    }
    class Demo{
        public static void main(String args[]) {
            MyFrame frame = new MyFrame();
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

LAYOUT MACHANISUMS- FLOWLAYOUT

```
import java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame {
    Label l1,l2;
    TextField tx1,tx2;
    Button b;
    MyFrame()
    {
        this.setVisible(true);
        this.setSize(340,500);
        this.setBackground(Color.green);
        this.setTitle("rattaiah");
        l1=new Label("user name:");
        l2=new Label("password:");
        tx1=new TextField(25);
        tx2=new TextField(25);
        b=new Button("login");
        tx2.setEchoChar('*');

        this.setLayout(new FlowLayout());
        this.add(l1);
        this.add(tx1);
        this.add(l2);
        this.add(tx2);
        this.add(b);
    }
}

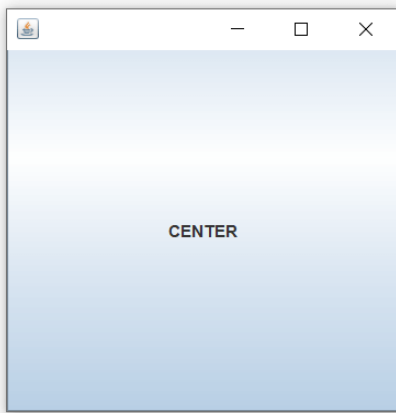
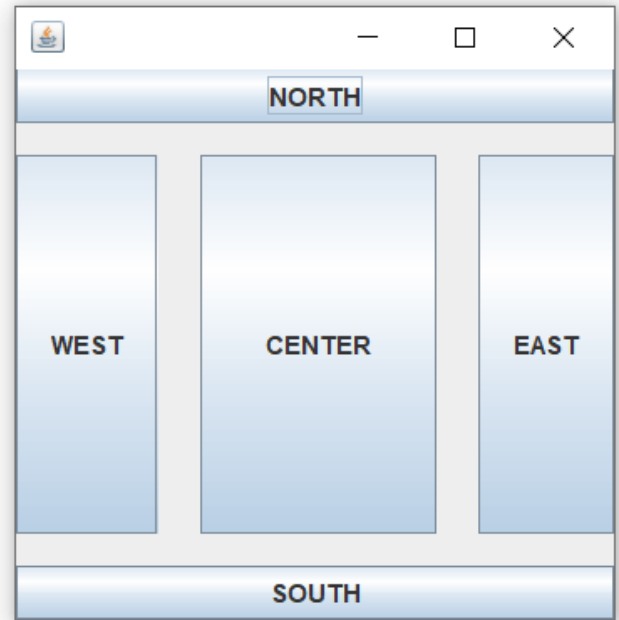
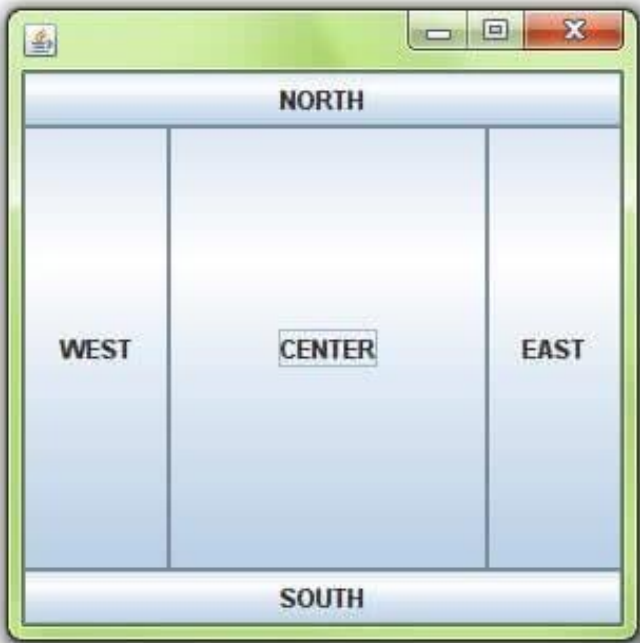
class Demo{
    public static void main(String[] args) {
        MyFrame f=new MyFrame();
    }
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

BORDERLAYOUT

```
import java.awt.*;
class MyFrame extends Frame {
    Button b1,b2,b3,b4,b5;
    MyFrame(){
        this.setBackground(Color.green);
        this.setSize(400,400);
        this.setVisible(true);
        this.setLayout(new BorderLayout());
        b1=new Button("Boys");
        b2=new Button("Girls");
        b3=new Button("management");
        b4=new Button("Teaching Staff");
        b5=new Button("non-teaching staff");
        this.add("North",b1);
        this.add("Center",b2);
        this.add("South",b3);
        this.add("East",b4);
        this.add("West",b5);
    }
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP



Dr. Naveen Kumar, Associate Professor,
AIIT, AUP



```
class Demo {  
public static void main(String[] args) {  
    MyFrame f=new MyFrame();  
}}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

CardLayout

```
import java.awt.*;
class MyFrame extends Frame {
    MyFrame() {
        this.setSize(400,400);
        this.setVisible(true);
        this.setLayout(new CardLayout());
        Button b1=new Button("button1");
        Button b2=new Button("button2");
        Button b3=new Button("button3");
        Button b4=new Button("button4");
        Button b5=new Button("button5");
        this.add("First Card",b1);
        this.add("Second Card",b2);
        this.add("Thrid Card",b3);
        this.add("Fourth Card",b4);
        this.add("Fifth Card",b5); } }
class Demo {
    public static void main(String args[]) {
        MyFrame frame = new MyFrame();
    }
```

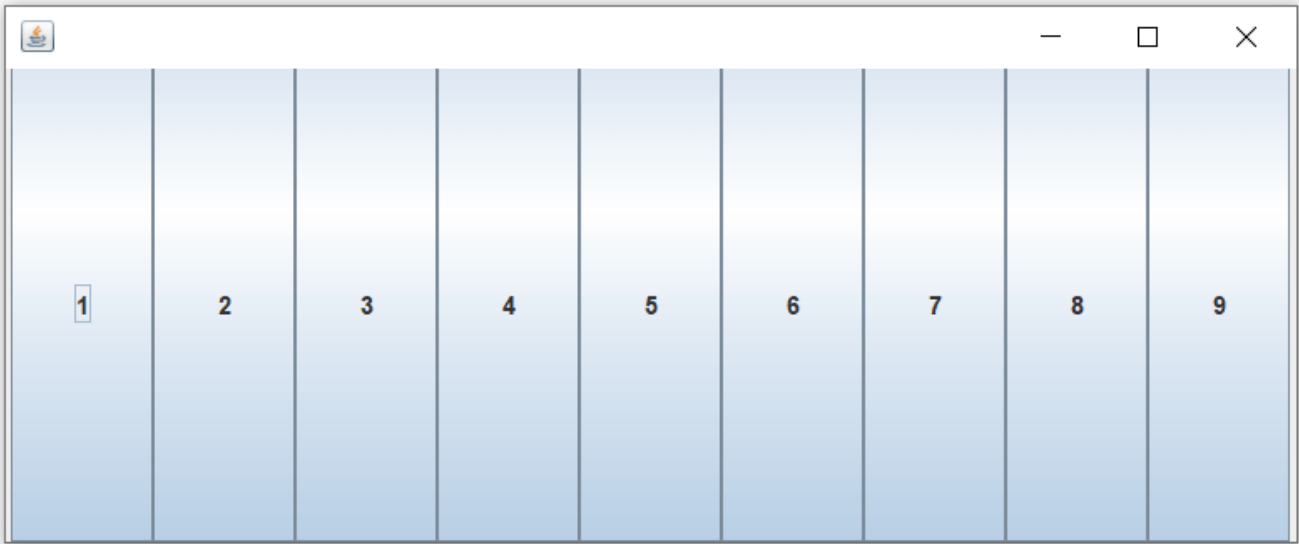
Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

GridLayout

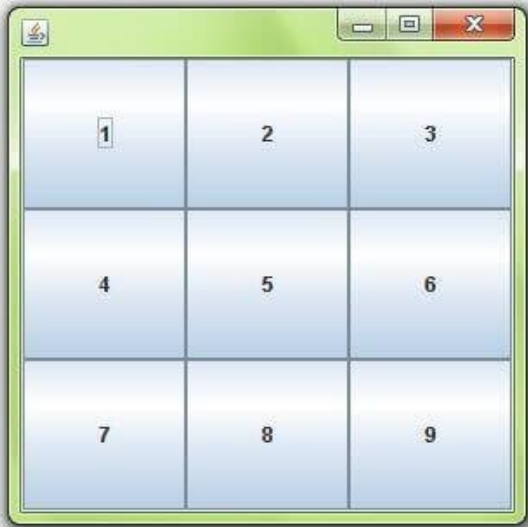
```
class MyFrame extends Frame
{
    MyFrame() {
        this.setVisible(true);
        this.setSize(500,500);
        this.setTitle("rattaiah");
        this.setBackground(Color.red);
        this.setLayout(new GridLayout(4,4));
        for (int i=0;i<10 ;i++ ){
            Button b=new Button(""+i);
            this.add(b);
        }
    }
}

class Demo {
    public static void main(String[] args) {
        MyFrame f=new MyFrame();
    }
}
```

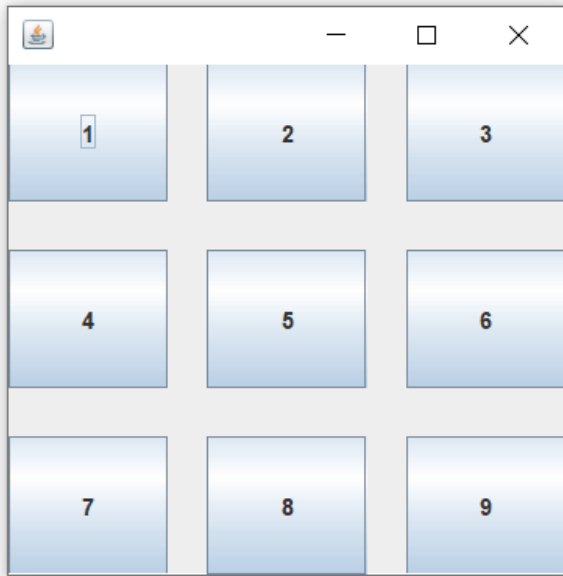
Dr. Naveen Kumar, Associate Professor,
AIIT, AUP



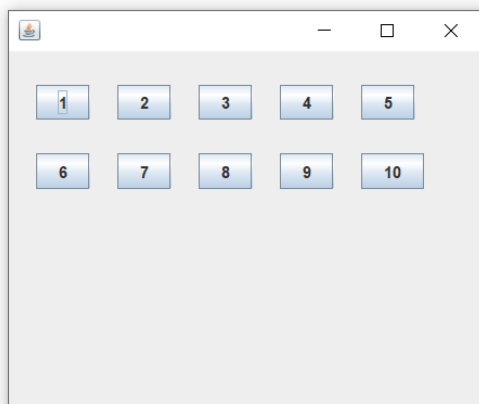
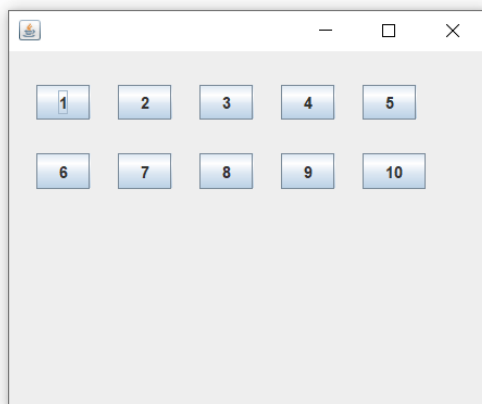
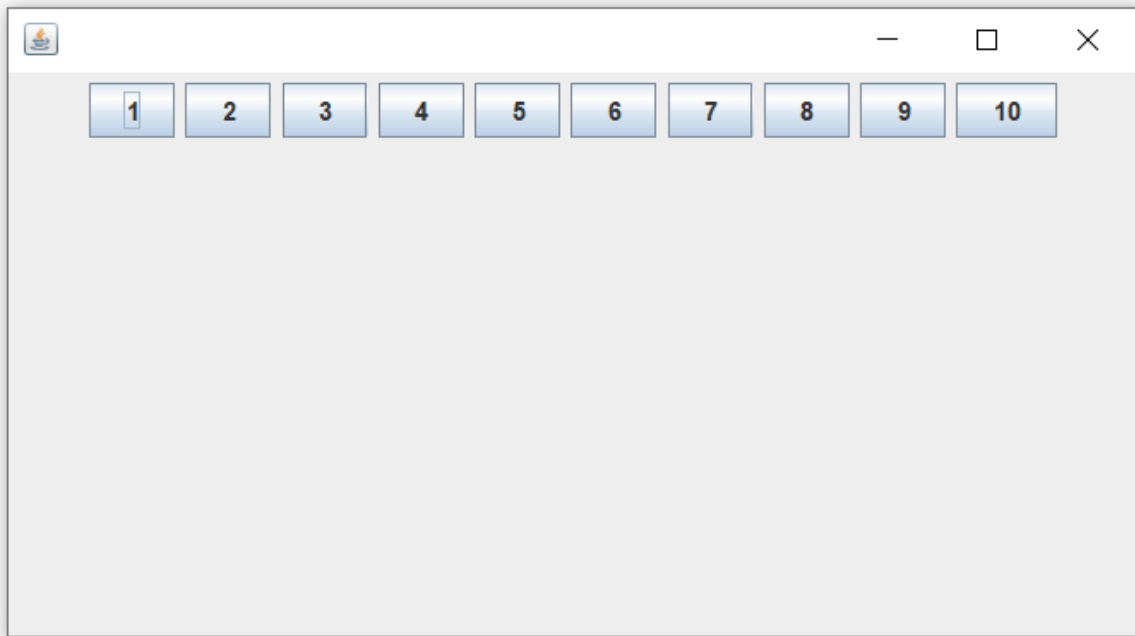
1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---



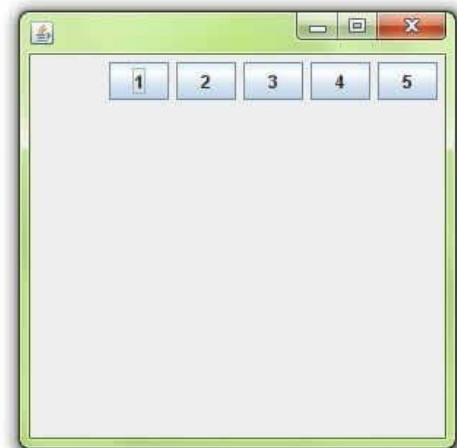
1	2	3
4	5	6
7	8	9



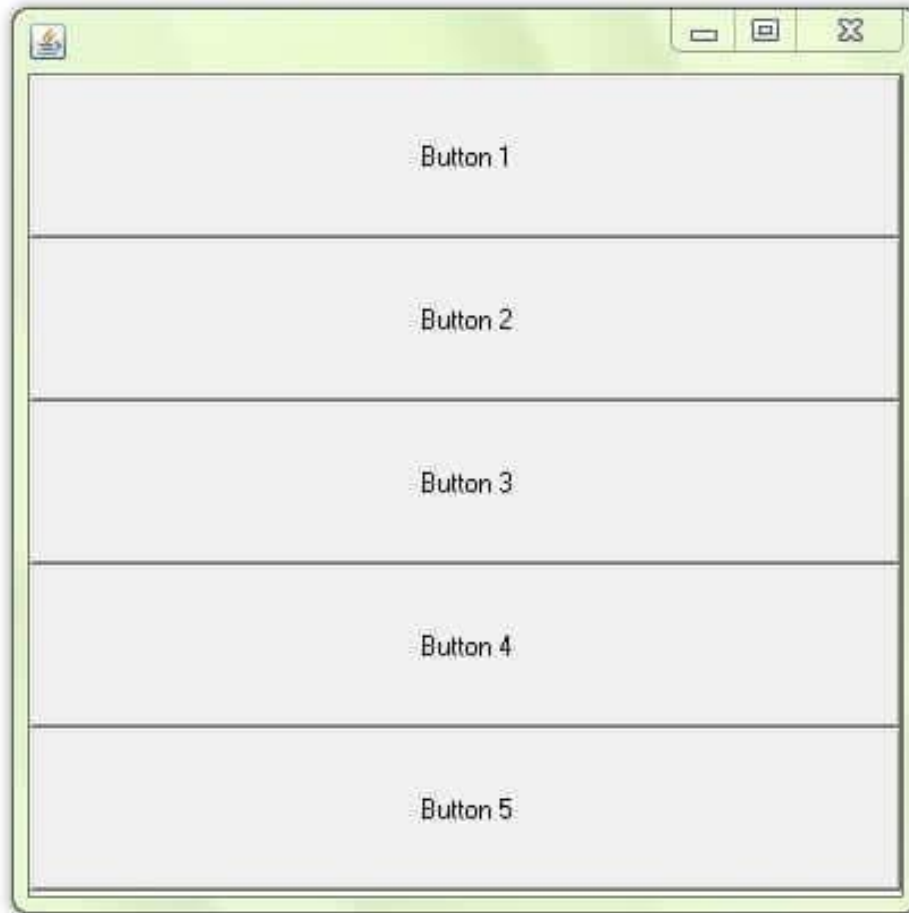
1	2	3
4	5	6
7	8	9



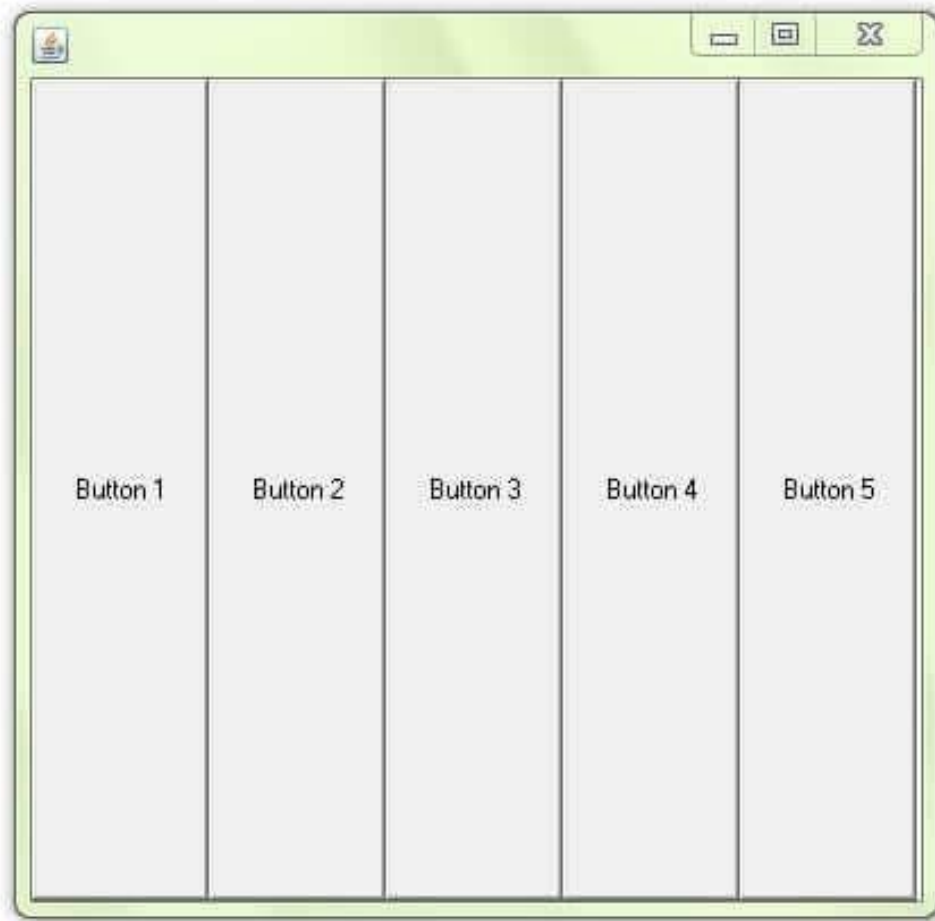
AIIT, AUP



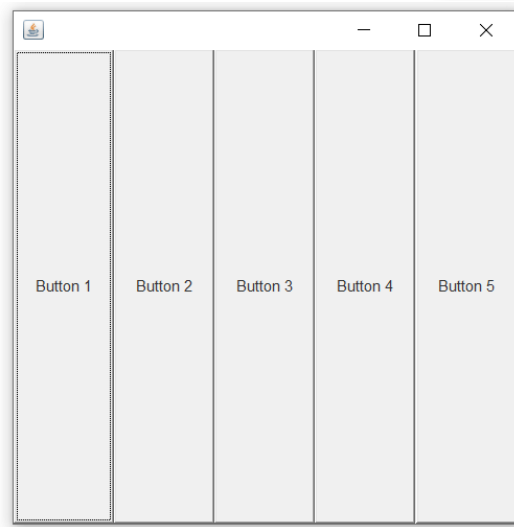
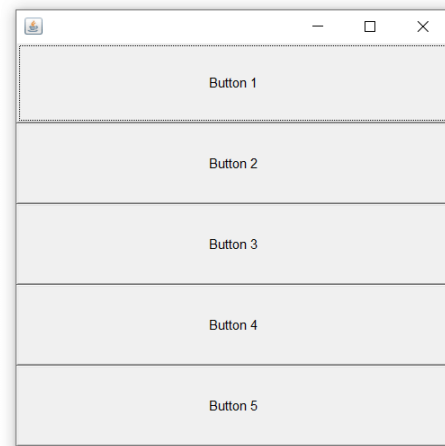
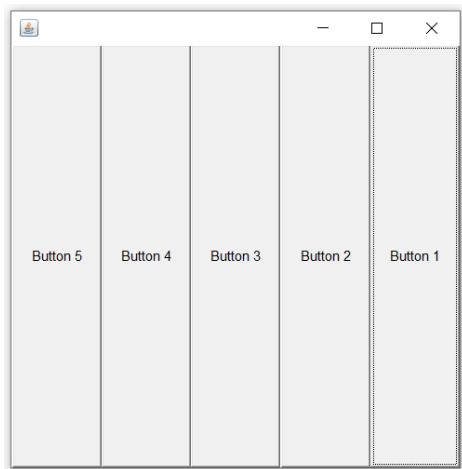
BoxLayout with X-AXIS



BoxLayout with LINE-AXIS



BoxLayout with PAGE-AXIS



Login Form

```
class MyFrame extends Frame implements ActionListener {
    Label l1,l2; TextField tx1,tx2; Button b;
    String status="";
    MyFrame()
    {
        setVisible(true);
        setSize(400,400);
        setTitle("girls");
        setBackground(Color.red);
        l1=new Label("user name:");
        l2=new Label("password:");
        tx1=new TextField(25);
        tx2=new TextField(25);
        b=new Button("login");
        b.addActionListener(this);
        tx2.setEchoChar('*');
        this.setLayout(new FlowLayout());
        this.add(l1);
        this.add(tx1);
        this.add(l2);
        this.add(tx2);
        this.add(b);
    }
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

Login Form

```
public void actionPerformed(ActionEvent ae) {  
    String uname=tx1.getText();  
    String upwd=tx2.getText();  
    if (uname.equals("navbharti")&&upwd.equals("aiitamity")) {  
        status="login success";  
    } else {  
        repaint();  
        status="login failure";  
    }  
    public void paint(Graphics g)  
    {  
        Font f=new Font("arial",Font.BOLD,30);  
        g.setFont(f); this.setForeground(Color.green);  
        g.drawString("Status:----"+status,50,300);  
    }  
}
```

MENUITEMS

- **import** java.awt.*;
- **import** java.awt.event.*;
- **class** MyFrame **extends** Frame **implements** ActionListener {
- String **label**="";
- MenuBar **mb**;
- Menu **m1,m2,m3**;
- MenuItem **mi1,mi2,mi3**;
- MyFrame()
- {
- **this**.setSize(300,300);
- **this**.setVisible(**true**);
- **this**.setTitle("myFrame");
- **this**.setBackground(Color.**green**);
- **mb=new** MenuBar();
- **this**.setMenuBar(**mb**);
- **m1=new** Menu("new");
- **m2=new** Menu("option");
- **m3=new** Menu("edit");
- **mb.add(m1); mb.add(m2);**
- **mb.add(m3);**

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

MENUITEMS

- `mi1=new JMenuItem("open");`
- `mi2=new JMenuItem("save");`
- `mi3=new JMenuItem("saveas");`
- `mi1.addActionListener(this);`
- `mi2.addActionListener(this);`
- `mi3.addActionListener(this);`
- `m1.add(mi1); m1.add(mi2);`
- `repaint(); }`
- **`public void`** `mouseReleased(MouseEvent e) {`
- `msg[2]="mouse released.....("+e.getX()+","+e.getY()+")";`
- `repaint(); }`
- **`public void`** `mouseEntered(MouseEvent e) {`
- `msg[3]="mouse entered.....("+e.getX()+","+e.getY()+")";`
- `repaint(); }`
- **`public void`** `mouseExited(MouseEvent e) {`
- `msg[4]="mouse exited.....("+e.getX()+","+e.getY()+")";`
- `repaint(); }`

MENUITEMS

- **public void** paint(Graphics g) {
- **int** X=50;
- **int** Y=100;
- **for**(**int** i=0;i<msg.length;i++) {
- **if** (msg[i]!=**null**) {
- g.drawString(msg[i],X,Y);
- Y=Y+50; }
- }
- }
- }
- **class** Demo {
- **public static void** main(String[] args) {
- MyFrame f=**new** MyFrame();
- }
- }

MENUITEMS

- **import** java.awt.*;
- **import** java.awt.event.*;
- **class** MyFrame **extends** Frame **implements** ActionListener {
- String **label**="";
- MenuBar **mb**;
- Menu **m1,m2,m3**;
- MenuItem **mi1,mi2,mi3**;
- MyFrame()
- {
- **this**.setSize(300,300);
- **this**.setVisible(**true**);
- **this**.setTitle("myFrame");
- **this**.setBackground(Color.**green**);
- **mb=new** MenuBar();
- **this**.setMenuBar(**mb**);
- **m1=new** Menu("new");
- **m2=new** Menu("option");
- **m3=new** Menu("edit");
- **mb.add(m1); mb.add(m2);**
- **mb.add(m3);**

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

Event handling

- It is possible to write the event handling code in following ways
 1. In different class.
 2. Same class
 3. By using anonymous inner classes.

- Steps to perform event handling:-
 - 1) Prepare the required components.
 - 2) Implements the listener interface override the methods to write the even handling code.
 - 3) Add the listener to the component.

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

Applying WindowListener on the Frame

- The Frame contains WindowListener it contains 7-methods listed below.
- **windowActivated(WindowEvent e)**
Invoked when the Window is set to be the active Window.
- **windowClosed(WindowEvent e)**
Invoked when a window has been closed as the result of calling dispose on the window.
- **windowClosing(WindowEvent e)**
Invoked when the user attempts to close the window from the window's system menu.
- **windowDeactivated(WindowEvent e)**
Invoked when a Window is no longer the active Window.
- **windowDeiconified(WindowEvent e)**
Invoked when a window is changed from a minimized to a normal state.
- **windowIconified(WindowEvent e)**
Invoked when a window is changed from a normal to a minimized state.
- **windowOpened(WindowEvent e)**
Invoked the first time a window is made visible.

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

- Note : if our class implements WindowListener interface we must override all the methods of windowListener interface . But if our class extends windowAdaptor class it is possible to override application required methods.

```
class MyFrame extends Frame { MyFrame()
```

- { this.setVisible(true); this.setSize(500,500);
- this.setBackground(Color.red); this.setTitle("MyFrame");
- this.addWindowListener(new Listenerimpl());
- } };
- class Listenerimpl extends WindowAdapter
- { public void windowClosing(WindowEvent we)
- { System.exit(0);
- } };
- class Demo
- { public static void main(String[] args)
- { MyFrame f=new MyFrame();
- } };

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

event handling in anonymous inner class

- `import java.awt.*;`
`import java.awt.event.*;`
`class MyFrame extends Frame { MyFrame()`
- `{ this.setVisible(true);`
- `this.setSize(500,500);`
- `} }`
- `this.setBackground(Color.red);`
- `this.setTitle("Myframe");`
- `this.addWindowListener(new WindowAdapter()`
`{ public void windowClosing(WindowEvent we)`
- `{ System.exit(0);`
- `} });`
- `class Demo`
`{ public static void main(String[] args)`
- `{ MyFrame f=new MyFrame();`
- `} };`

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

ACTIONLISTENER

- The below example we are performing addition and multiplications when we click add & mul buttons. So whenever we clicking button the button is able to listen the event to do this add the Listener to button.
The appropriate listener for button is ActionListener.

- `public void actionPerformed(ActionEvent e) {`
- `try{`
- `int fval=Integer.parseInt(tx1.getText());`
- `int sval=Integer.parseInt(tx2.getText());`
- `String label=e.getActionCommand();`
`if (label.equals("add"))`
- `{ result=fval+sval; }`
`if (label.equals("mul"))`
`{ result=fval*sval; }`
- `tx3.setText(""+result);`
`}`
`catch(Exception ee)`
`{ ee.printStackTrace(); }`

Login Form

- **class** MyFrame **extends** Frame **implements** ActionListener {
- Label l1,l2;
- TextField tx1,tx2; Button b;
- String status="";
- MyFrame(){
- setVisible(**true**);
- setSize(400,400);
- setTitle("girls");
- setBackground(Color.red);
- l1=**new** Label("user name:");
- l2=**new** Label("password:");
- tx1=**new** TextField(25);
- tx2=**new** TextField(25);
- b=**new** Button("login");
- b.addActionListener(**this**);
- tx2.setEchoChar('*');
- **this**.setLayout(**new** FlowLayout());
- **this**.add(l1);
- **this**.add(tx1);
- **this**.add(l2);

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

Login Form

- **this**.add(tx2);
- **this**.add(b);
- }
- **public void** actionPerformed(ActionEvent ae) {
- String uname=tx1.getText();
- String upwd=tx2.getText();
- **if** (uname.equals("Sravya")&&upwd.equals("dss")){
- status="login success";
- } **else** {
- status="login failure";
- }
- repaint();
- }
- **public void** paint(Graphics g)
- {
- Font f=**new** Font("arial",Font.BOLD,30);
- g.setFont(f);
- **this**.setForeground(Color.green);
- g.drawString("Status:----"+status,50,300);
- }

Login Form

- **class** Demo
- { **public static void** main(String[] args)
- { MyFrame f=**new** MyFrame();
- }}

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

MenuItem

- **class** MyFrame **extends** Frame **implements** ActionListener {
- String label="";
- MenuBar mb;
- Menu m1,m2,m3;
- MenuItem mi1,mi2,mi3;
- MyFrame()
- { **this**.setSize(300,300);
- **this**.setVisible(**true**);
- **this**.setTitle("myFrame");
- **this**.setBackground(Color.green);
- mb=**new** MenuBar();
- **this**.setMenuBar(mb);
- m1=**new** Menu("new");
- m2=**new** Menu("option");
- m3=**new** Menu("edit");
- mb.add(m1);
- mb.add(m2);
- mb.add(m3);
- mi1=**new** MenuItem("open");
- mi2=**new** MenuItem("save");

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

MenuItem

- `mi3=new JMenuItem("saveas");`
- `mi1.addActionListener(this);`
- `mi2.addActionListener(this);`
- `mi3.addActionListener(this);`
- `m1.add(mi1);`
- `m1.add(mi2);`
- `m1.add(mi3);`
- `}`
- **public void** `actionPerformed(ActionEvent ae) {`
- `label=ae.getActionCommand();`
- `repaint();`
- `}`
- **public void** `paint(Graphics g) {`
- `Font f=new Font("arial",Font.BOLD,25);`
- `g.setFont(f);`
- `g.drawString("Selected item....."+label,50,200);`
- `}`
- `}`
- **class** `Demo{`
- **public static void** `main(String[] args) {`
- `MyFrame f=new MyFrame();`
- `}`
- `}`

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

MouseListener

- **class** myframe **extends** Frame **implements** MouseListener {
- String[] **msg**=**new** String[5];
- myframe()
- { **this**.setSize(500,500);
- **this**.setVisible(**true**);
- **this**.addMouseListener(**this**);
- }
- **public void** mouseClicked(MouseEvent e)
- { **msg**[0]="mouse clicked.....("+e.getX()+",""+e.getY()+)";
- repaint(); }
- **public void** mousePressed(MouseEvent e)
- { **msg**[1]="mouse pressed.....("+e.getX()+",""+e.getY()+)";
- repaint(); }
- **public void** mouseReleased(MouseEvent e)
- { **msg**[2]="mouse released.....("+e.getX()+",""+e.getY()+)";
- repaint(); }
- **public void** mouseEntered(MouseEvent e)
- { **msg**[3]="mouse entered.....("+e.getX()+",""+e.getY()+)";
- repaint(); }
- **public void** mouseExited(MouseEvent e)

MouseListener

- { msg[3]="mouse entered.....("+e.getX()+","+e.getY()+");
- repaint(); }
- **public void** mouseExited(MouseEvent e)
- { msg[4]="mouse exited.....("+e.getX()+","+e.getY()+");
- repaint(); }
- **public void** paint(Graphics g) { **int** X=50;
- **int** Y=100;
- **for**(**int** i=0;i<msg.length;i++) {
- **if** (msg[i]!=null)
- { g.drawString(msg[i],X,Y); Y=Y+50;
- }
- }}}
- **class** Demo
- { **public static void** main(String[] args)
- { myframe f=**new** myframe();
- }

ItemListener

- **import** java.awt.*;
- **import** java.awt.event.*;
- **class** myframe **extends** Frame **implements** ItemListener {
- String qual="",gen="";
- Label l1,l2;
- CheckboxGroup cg;
- Checkbox c1,c2,c3,c4,c5;
- Font f;
- myframe()
- { **this**.setSize(300,400);
- **this**.setVisible(**true**);
- **this**.setLayout(**new** FlowLayout());
- l1=**new** Label("Qualification: ");
- l2=**new** Label("Gender: ");
- c1=**new** Checkbox("BSC");
- c2=**new** Checkbox("BTECH");
- c3=**new** Checkbox("MCA");
- cg=**new** CheckboxGroup();
- c4=**new** Checkbox("Male",cg,**false**);
- c5=**new** Checkbox("Female",cg,**true**);

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

ItemListener

- c1.addItemListener(**this**);
- c2.addItemListener(**this**);
- c3.addItemListener(**this**);
- c4.addItemListener(**this**);
- c5.addItemListener(**this**);
- **this**.add(l1); **this**.add(c3); **this**.add(c5);
- **this**.add(c1); **this**.add(l2);
- **this**.add(c2); **this**.add(c4);
- **public void** itemStateChanged(ItemEvent ie) {
- **if**(c1.getState()==**true**)
- { **qual**=**qual**+c1.getLabel()+" "; }
- **if**(c2.getState()==**true**)
- { **qual**=**qual**+c2.getLabel()+" "; }
- **if**(c3.getState()==**true**)
- { **qual**=**qual**+c3.getLabel()+" "; }
- **if**(c4.getState()==**true**)
- { **gen**=c4.getLabel(); }
- **if**(c5.getState()==**true**)
- { **gen**=c5.getLabel(); }
- repaint();

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

ItemListener

- **public void** paint(Graphics g)
- { Font f=**new** Font("arial",Font.**BOLD**,20);
- g.setFont(f);
- **this**.setForeground(Color.**green**);
- g.drawString("qualification----->" + qual,50,100);
- g.drawString("gender----->" + gen,50,150);
- qual="";
- gen="";
- }
- }
- **class** ItemListenerDemo
- { **public static void** main(String[] args)
- { myframe f=**new** myframe();
- } }

KeyListener

- **class** myframe **extends** Frame
- {
- myframe()
- { **this**.setSize(400,400);
- **this**.setVisible(**true**);
- **this**.setBackground(Color.**green**);
- **this**.addKeyListener(**new** keyboardimpl());
- }
- }
- **class** keyboardimpl **implements** KeyListener {
- **public void** keyTyped(KeyEvent e){
- System.**out**.println("key typed "+e.getKeyChar());
- }
- **public void** keyPressed(KeyEvent e){
- System.**out**.println("key pressed "+e.getKeyChar());
- }
- **public void** keyReleased(KeyEvent e){
- System.**out**.println("key released "+e.getKeyChar());
- }
- }
- **class** Demo
- { **public static void** main(String[] args)
- { myframe f=**new** myframe();
- }}

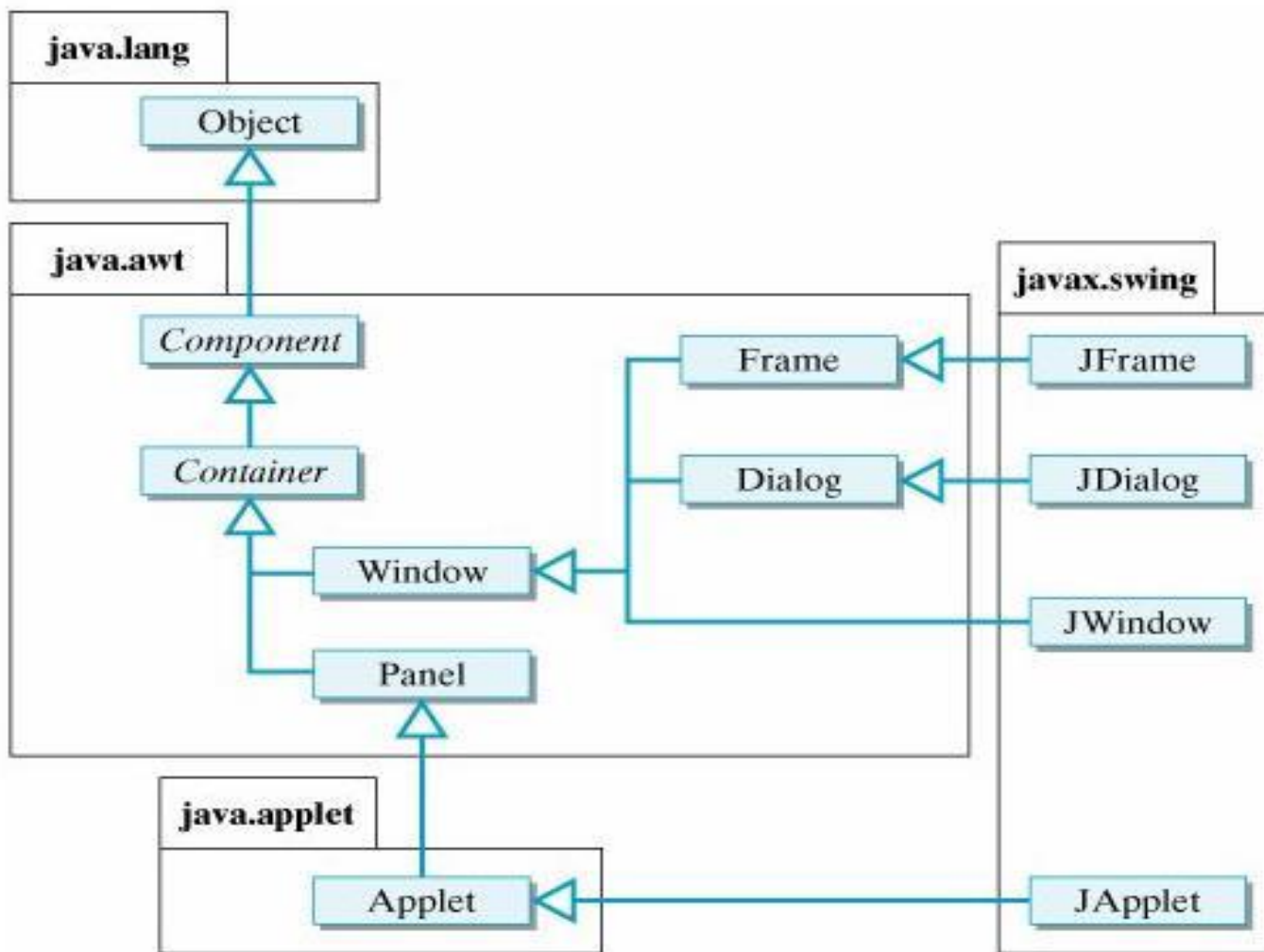
Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

Swings

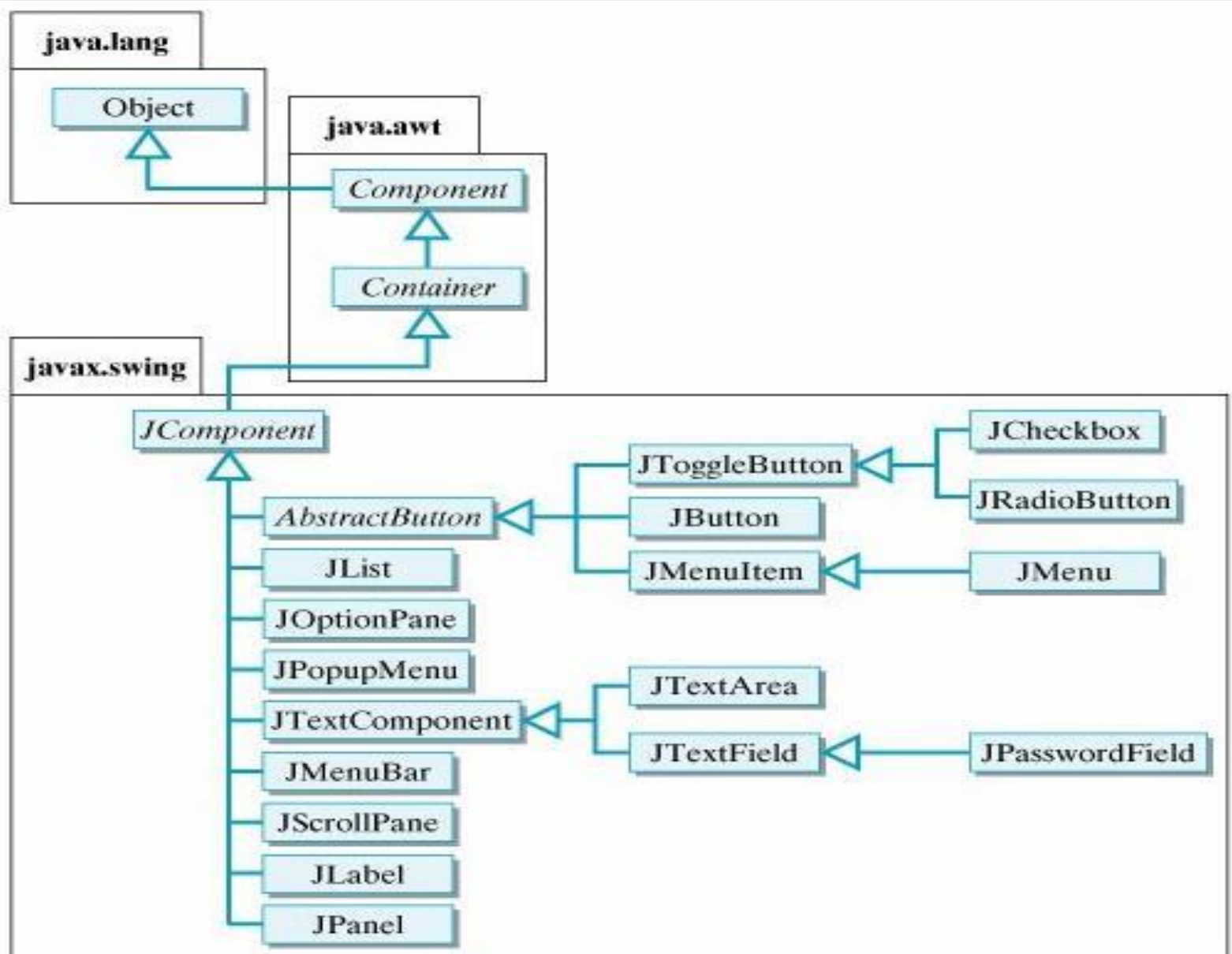
1. Sun Micro Systems introduced **AWT** to prepare GUI applications but awt components not satisfy the client requirement.
2. An alternative to AWT **Netscape** Communication Corporation has provided set of GUI components in the form of **IFC**(Internet Foundation Class) but IFC also **provide less performance** and it is not satisfy the client requirement.
3. In the above context[**sun & Netscape**] combine and introduced common product to design GUI applications is called **JFS**(java foundation classes) then it is renamed as **swings**.

AWT vs Swings

- AWT components are **platform dependent** but Swings components are **platform independent** because these components are completely written in java .
- AWT components are **heavyweight** component but swing components are **light weight** component.
- AWT components **consume more number of system resources** Swings consume **less number of system resources**.
- AWT is provided **less number of components** whereas a swing provides **more number of components**.
- AWT doesn't provide **Tooltip** Test support but swing components have **provided Tooltip** test support.
- In awt to close the window : **windowListener windowAdaptor** In case of swing use **small piece of code**. `f.setDefaultCloseOperation(JFrame.EXIT-ON-CLOSE);`
- In case of AWT we will add the GUI components in the Frame directly but Swing we will add all GUI components to panes to accommodate GUI components.
- The awt classes & interfaces are present in java.awt packages & swing classes are present in **javax.swing** package.



Dr. Naveen Kumar, Associate Professor,
AIIT, AUP



JFrame Demo

```
public class MyFrame extends JFrame{
    public MyFrame() {
        setVisible(true);
        setSize(300, 300);
        setTitle("MyFrame");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new FlowLayout());
        JLabel label = new JLabel("Hello World:");
        JButton button = new JButton("click me");
        add(label);
        add(button);
    }
    public static void main(String[] args) {
        MyFrame f = new MyFrame();
    }
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

SWING GUI COMPONENTS EXAMPLE

```
class JFrame1Demo extends JFrame{
    JLabel l1,l2,l3,l4,l5,l6,l7;
    JTextField tf;
    JPasswordField pf;
    JCheckBox cb1,cb2,cb3;
    JRadioButton rb1,rb2;
    JList l;
    JComboBox cb;
    JTextArea ta;
    JButton b;
    Container c;
    JFrame1Demo()
    {
        setVisible(true);
        setSize(150,500);
        setTitle("SWING GUI COMPONENTS EXAMPLE");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new FlowLayout());
        getContentPane().setBackground(Color.red);
        setBackground(Color.green);
    }
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP


```
l1=new JLabel("User Name");
l2= new JLabel("password");
l3= new JLabel("Qualification");
l4= new JLabel("User Gender");
l5= new JLabel("Technologies");
l6= new JLabel("UserAddress");
l7= new JLabel("comments");
tf=new JTextField(15);
tf.setToolTipText("TextField");
pf=new JPasswordField(15);
pf.setToolTipText("PasswordField");
cb1=new JCheckBox("BSC",false);
cb2=new JCheckBox("MCA",false);
cb3=new JCheckBox("PHD",false);
rb1=new JRadioButton("Male",false);
rb2=new JRadioButton("Female",false);
ButtonGroup bg=new ButtonGroup();
bg.add(rb1);
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

```
bg.add(rb2);
String[] listitems={"cpp","c","java"};
l=new JList(listitems);
String[] cbitems={"hyd","pune","bangalore"};
cb=new JComboBox(cbitems);
ta=new JTextArea(5,20);
b=new JButton("submit");
add(l1);
add(tf);
add(l3);
add(l4);
add(l);
add(ta);
add(l2);
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

```
add(pf);
add(cb1);
add(cb2);
add(cb3);
add(rb1);
add(rb2);
add(l5);
add(l6);
add(cb);
add(l7);
add(b);
}
public static void main(String[] args) {
JFrame1Demo f=new JFrame1Demo();
}
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

ImageIcon in Swings

```
public class MyFrame extends JFrame{
    public MyFrame() {
        setVisible(true);
        setSize(200, 200);
        setTitle("MyFrame");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout()); //setting layout to frame
        JLabel label = new JLabel(new ImageIcon("download.jpg"));
        add(label);
        label.setLayout(new FlowLayout()); // setting layout to back ground image
        JLabel label2 = new JLabel("Hello World:");
        JButton button = new JButton("click me");
        label.add(label2);
        label.add(button);
    }
    public static void main(String[] args) {
        MyFrame f = new MyFrame();
    }
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

MessageDialog in Swings

```
public class MyFrame2 extends JFrame implements ActionListener {
    JRadioButton button1, button2;
    JButton button;
    public MyFrame2() {
        setVisible(true);
        setTitle("MyFrame");
        setSize(300, 300);
        setLayout(new FlowLayout());
        button1 = new JRadioButton("Male");
        button2 = new JRadioButton("Female");
        button = new JButton("click");
        ButtonGroup group = new ButtonGroup();
        group.add(button1);
        group.add(button2);
        add(button1);
        add(button2);
        add(button);
        button.addActionListener(this);
    }
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

MessageDialog in Swings

@Override

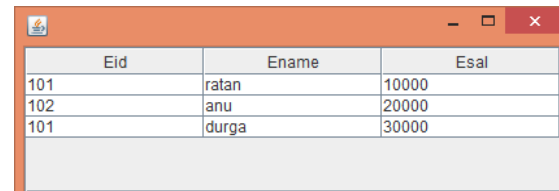
```
public void actionPerformed(ActionEvent e) {  
    if(button1.isSelected())  
    { JOptionPane.showMessageDialog(this, "U r selected Male option thankyou");}  
    if(button2.isSelected())  
    { JOptionPane.showMessageDialog(this, "U r selected Female option thankyou");}  
}  
  
public static void main(String[] args) {  
    MyFrame2 frame2 = new MyFrame2();  
}  
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

JTable Demo

```
public class JFrame4Demo extends JFrame {  
    public JFrame4Demo() {  
        setVisible(true);  
        setSize(400, 200);  
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        String[][]  
        data={{"101","rakesh","10000"},"102","Sita","20000"},"101","Rajeev","30000"}};  
        String[] col={"Eid","Ename","Esal"};  
        JTable jTable = new JTable(data,col);  
        add(jTable);  
        JScrollPane jScrollPane = new JScrollPane(jTable); add(jScrollPane);  
    }  
    public static void main(String[] args)  
    {  
        JFrame4Demo frame3 = new JFrame4Demo();  
    }  
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP



Eid	Ename	Esal
101	rakesh	10000
102	anu	20000
101	durga	30000

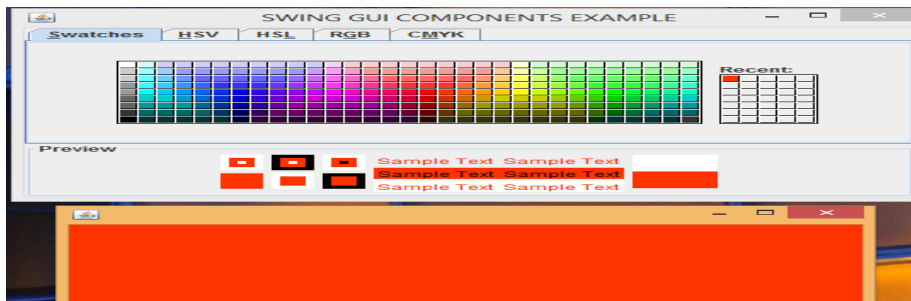
JColorChooser

```
class JFrame5 extends JFrame implements ChangeListener { JColorChooser cc;  
Container c;  
JFrame5(){  
    this.setVisible(true);  
    this.setSize(500,500);  
    this.setTitle("SWING COLORCHOOSER EXAMPLE");  
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    c=getContentPane();  
    cc=new JColorChooser();  
    cc.getSelectionModel().addChangeListener(this);  
    c.add(cc);  
}  
public void stateChanged(ChangeEvent c) {  
    Color color=cc.getColor();  
    JFrame f=new JFrame();  
    f.setSize(400,400);  
    f.setVisible(true);  
    f.getContentPane().setBackground(color);  
}  
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

JColorChooser

```
public class ColorChooserDemo{  
    public static void main(String[] args)  
    {  
        JFrame5 f=new JFrame5();  
    }  
}
```



Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

JFileChooser

```
public class JFileChooserDemo {  
    public static void main(String[] args){  
        JFrame6 f=new JFrame6();  
    }  
    class JFrame6 extends JFrame implements ActionListener{  
        JFileChooser fc;  
        Container c;  
        JLabel l;  
        JTextField tf;  
        JButton b;  
        JFrame6(){  
            this.setVisible(true);  
            this.setSize(500,500);  
            this.setTitle("SWING GUI COMPONENTS EXAMPLE");  
            this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
            c=getContentPane();  
            l=new JLabel("Select File:");  
            tf=new JTextField(25);  
            b=new JButton("BROWSE");  
            this.setLayout(new FlowLayout());  
            b.addActionListener(this);  
            c.add(l);  
            c.add(tf);  
            c.add(b);  
        }  
    }  
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP

JFileChooser

```
public void actionPerformed(ActionEvent ae)
{
    class FileChooser1 extends JFrame implements ActionListener
    {
        FileChooser1()
        {
            Container c=getContentPane();
            this.setVisible(true); this.setSize(500,500);
            fc=new JFileChooser(); fc.addActionListener(this); fc.setLayout(new FlowLayout()); c.add(fc);
        }
        public void actionPerformed(ActionEvent ae) {
            File f=fc.getSelectedFile();
            String path=f.getAbsolutePath();
            tf.setText(path);
            this.setVisible(false);
        }
    }
    new FileChooser1();
}
```

Dr. Naveen Kumar, Associate Professor,
AIIT, AUP



THANK YOU

Dr. Naveen Kumar, Associate Professor, AIIT, AUP